

Project 2

Sentiment Analysis & Review Classification System (NLP-Based End-to-End Project)

Project Overview:

This major project helps students build a full **Natural Language Processing (NLP)** pipeline where the model can understand emotions, opinions, and sentiments expressed in text data. Students also learn how to clean text, train NLP models, deploy them via APIs, and create a functioning AI web service.

Project Objective:

Build an end-to-end **Sentiment Analysis System** that can classify text (reviews, comments, tweets, feedback) into categories such as **Positive, Negative, or Neutral**.

Learn Fast, Understand Better.

Recommended Datasets:

- Twitter Sentiment Dataset
 - IMDb Movie Reviews
 - Amazon Product Reviews
 - Any labeled text dataset provided by Enginow
-

Key Concepts Students Will Learn:

- Text cleaning and NLP preprocessing
- Tokenization, stemming, lemmatization
- Vectorization: Bag of Words, TF-IDF, word embeddings
- Training ML models (Logistic Regression, Naive Bayes) or DL models (LSTM, GRU)
- Fine-tuning hyperparameters
- Building prediction pipelines
- Creating APIs using Flask / FastAPI
- Model deployment basics
- Writing proper documentation
- Making the project portfolio-ready

Workflow Breakdown:

1. Data Cleaning & Preprocessing

- Remove punctuation, numbers, special characters
- Normalize text (lowercase, lemmatization)
- Remove stopwords
- Tokenize the dataset
- Convert text into numerical vectors (TF-IDF or embeddings)

2. Model Development

Choose between:

- **Classical ML Model:** Logistic Regression, SVM, Naive Bayes

- **Deep Learning Model:** LSTM or Bi-LSTM for high accuracy

Students experiment with both to compare performance.

3. Evaluation & Metrics

- Accuracy, Precision, Recall, F1-score
- Confusion matrix visualization
- Model comparison report

4. Exporting the Model

- Save trained model in `.pkl` or `.h5` format
- Save preprocessing pipeline separately

5. API Development (MLOps Component)

Using Flask or FastAPI:

- Build `/predict` endpoint
- Load saved model + preprocessing
- Accept input text via POST/GET
- Return predicted sentiment

6. Optional Frontend (Bonus)

Create a simple UI using HTML/CSS or Streamlit where users can input text and view sentiment predictions in real-time.

7. Documentation & GitHub Repository

Include:

- README file with project overview

- Step-by-step instructions
- Screenshots and results
- API testing screenshots

8. Final Deliverables

- Fully working model
- API endpoint
- Project code + README on GitHub
- Final report (8–10 lines summarizing project and learnings)

Outcome:

Students learn how AI models for language processing work end-to-end. They complete a deployable NLP project suitable for internships, placements, GitHub portfolios, and resume enhancement.

This project demonstrates:

- Technical depth
- Understanding of ML + NLP workflows
- Ability to build real-world AI applications
- API and deployment readiness